

Iterator Stream

Lecture 08 + Tutorial 09

2.1 Iterator

Iterator

- `boolean hasNext()`
- `E next()`

“ “ “ “

2.2

`[start, end)`

```
class RangeIterator {
    RangeIterator(int start, int end) {
        this.start = start;
        this.end = end;
        assert this.start <= this.end;
    }

    boolean hasNext() {
        return this.start < this.end;
    }

    int next() {
        int current = this.start;
        this.start++;
        return current;
    }

    private int start;
    private int end;
}
```

- `start` `end`
-
- `hasNext()`
- `next()` `start` 1

```
RangeIterator it = new RangeIterator(10, 14);
System.out.println(it.hasNext()); // 输出 true
System.out.println(it.next());    // 输出 10
// 依次调用next会返回11、12、13, 之后hasNext返回false
```

2.3

```
int sum = 0;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        for (int k = 0; k < p; k++) {
            sum += matrix.get(i, j, k);
        }
    }
}
```

BoxIterator

```
BoxIterator it = new BoxIterator(n, m, p);
int sum = 0;
while (it.hasNext()) {
    Point x = it.next();
    sum += matrix.get(x.i, x.j, x.k);
}
```

2.4

2.4.1

Linked List

```
class List<T> {
    private class Node {
        T elem;
        Node next;
    }
    private Node start;

    ListIterator iterator() {
        return new ListIterator(this.start);
    }
}
```

```

    }

    class ListIterator {
        private ListIterator(Node p) {
            this.current = p;
        }
        boolean hasNext() {
            return this.current != null;
        }
        T next() {
            T elem = this.current.elem;
            this.current = this.current.next;
            return elem;
        }
        private Node current;
    }
}

```

- ListIterator List List Node
- current
- hasNext
- next() current
- List iterator()

2.4.2

Binary Tree

```

class BinaryTreeIterator<T> {
    private Stack<BinaryTree<T>> stack;

    BinaryTreeIterator(BinaryTree<T> tree) {
        this.stack = new Stack<BinaryTree<T>>();
        if (tree != null) {
            this.stack.push(tree);
        }
    }

    boolean hasNext() {
        return this.stack.size() > 0;
    }

    T next() {
        BinaryTree<T> t = this.stack.pop();
        if (t.left != null) {
            this.stack.push(t.left);
        }
        if (t.right != null) {
            this.stack.push(t.right);
        }
    }
}

```

```

    }
    return t.elem;
}
}

```

-
- `hasNext`
- `next()`

2.5 Stream

Stream /

-
-
-

" "

Fibonacci

```

class FibonacciIterator {
    FibonacciIterator() {
        this.a = 0;
        this.b = 1;
    }
    int next() {
        int current = this.a;
        this.a = this.b;
        this.b = current + this.b;
        return current;
    }
    private int a;
    private int b;
}

```

`hasNext`

`true`

`next()`

2.6 Tutorial 09

2.6.1

`0, 2, 4, 6...`

```

class EvenIterator implements Iterator<Integer> {
    private int current = 0;

    @Override
    public boolean hasNext() {
        return true;
    }

    @Override
    public Integer next() {
        int result = current;
        current += 2;
        return result;
    }
}

```

Java Iterator<Integer> current 0 next 2

hasNext

2.6.2

```

class ReverseArrayIterator<T> implements Iterator<T> {
    private T[] array;
    private int index;

    public ReverseArrayIterator(T[] array) {
        this.array = array;
        this.index = array.length - 1;
    }

    @Override
    public boolean hasNext() {
        return index >= 0;
    }

    @Override
    public T next() {
        return array[index--];
    }
}

```

- index
- hasNext 0
- next 1

```
class FibonacciIterator implements Iterator<Integer> {
    private int prev = 0;
    private int curr = 1;

    @Override
    public boolean hasNext() {
        return true;
    }

    @Override
    public Integer next() {
        int next = prev + curr;
        prev = curr;
        curr = next;
        return prev;
    }
}
```

1.	Lexer/Tokenizer	Token	"35 + 42 * 2"
	[数字35, 加号, 数字42, 乘号, 数字2]		
2.	Parser	Token	AST
3.	Evaluator/Interpreter	AST	

6 / 21



**** File ****
Byte 8

3.2

-
-
-
-

3.3 Java

3.3.1

FileInputStream

```
import java.io.*;
public class Main {
    public static void main(String[] args) throws IOException {
        FileInputStream fis = new FileInputStream("a.txt");
        System.out.println(fis.read());
        fis.close();
    }
}
```

- FileInputStream
- read() int EOF -1 0~255
- close()

3.3.2 Exception

IO			Java
FileInputStream	read	close	IOException

1. try-catch

```
try {
    FileInputStream fis = new FileInputStream("a.txt");
    System.out.println(fis.read());
    fis.close();
} catch (IOException e) {
    System.out.println("Could not read the file.");
}
```

2. **throws** throws IOException

3.3.3

```
ArrayList<Byte> bytes = new ArrayList<Byte>();
FileInputStream fis = new FileInputStream("a.txt");
while (true) {
    int next = fis.read();
    if (next == -1) {
        break;
    }
    bytes.add((byte)next);
}
fis.close();
```

read -1 byte

3.4 Encoding

- ASCII 0~127
- UTF-8 ASCII 3
- GBK Big5

Java InputStreamReader

```
FileInputStream fis = new FileInputStream("nihao.txt");
InputStreamReader isr = new InputStreamReader(fis, "UTF-8");
while (true) {
    int next = isr.read();
    if (next == -1) break;
    System.out.println((char)next);
}
```



```
}
isr.close();
```

InputStreamReader
char

UTF-8

read()

3.5

3.5.1

FileOutputStream

```
import java.io.*;
public class Main {
    public static void main(String[] args) throws IOException {
        FileOutputStream fos = new FileOutputStream("a.txt");
        fos.write(65);
        fos.write(66);
        fos.write(67);
        fos.close();
    }
}
```

65 66 67

ASCII 'A' 'B' 'C'

3.5.2

OutputStreamWriter

```
FileOutputStream fos = new FileOutputStream("a.txt");
OutputStreamWriter osw = new OutputStreamWriter(fos, "UTF-8");
osw.write("你好");
osw.close();
```

UTF-8

3.6 Serialization

-
-

3.6.1

1234

"1234;"

```
int readInt(FileInputStream fis) throws IOException {
    int n = 0;
    while (true) {
        int next = fis.read();
        if (next == ';') {
            break;
        }
        int digit = next - '0';
        n = 10 * n + digit;
    }
    return n;
}
```

$$n = 10 * n + \text{digit}$$

```
void writeInt(FileOutputStream fos, int n) throws IOException {
    int p = 1;
    while (10 * p <= n) {
        p = 10 * p;
    }
    while (p >= 1) {
        fos.write('0' + (n / p));
        n = n % p;
        p = p / 10;
    }
    fos.write(';');
}
```

$$p$$

3.6.2

$$[123, 456, 789]$$

"3;123;456;789;"

Tutorial 09 Reversi

- 当前玩家 + 棋盘大小 + 棋盘内容
B8;EEEEEEEEEEEEEEEEEEEEWWEEEEWWBBEEWWBBEEEEBBBBBBEEEE
- B 8 8x8 E B W

IO

Lecture 10 + Tutorial 10

4.1 Buffering

4.1.1

`read()/write()`

Buffer

4.1.2

```
FileInputStream fis = new FileInputStream("a.txt");
byte[] buffer = new byte[BUFFER_SIZE];
fis.read(buffer);
fis.close();
```

`read(byte[] buffer)` `buffer.length`
-1

```
int add_all_bytes(String filename) throws IOException {
    FileInputStream fis = new FileInputStream(filename);
    byte[] buffer = new byte[BUFFER_SIZE];
    int sum = 0;
    int n;
    while ((n = fis.read(buffer)) != -1) {
        for (int i = 0; i < n; i++) {
            sum += buffer[i];
        }
    }
    fis.close();
    return sum;
}
```

`n` `BUFFER_SIZE`

1KB 8KB 64KB

4.1.3

```
FileOutputStream fos = new FileOutputStream("a.txt");
byte[] buffer = new byte[BUFFER_SIZE];
// 填充缓冲区数据
fos.write(buffer);
fos.close();
```

`write(byte[] buffer, int offset, int length)`

```
void write_random(String filename, int n) throws IOException {
    byte[] buffer = new byte[BUFFER_SIZE];
    FileOutputStream fos = new FileOutputStream(filename);
    Random random = new Random();
    int remaining = n;
    while (remaining > 0) {
        random.nextBytes(buffer);
        int written = Math.min(remaining, BUFFER_SIZE);
        fos.write(buffer, 0, written);
        remaining = remaining - written;
    }
    fos.close();
}
```

`n`

4.2 Random Access

RandomAccessFile

- `new RandomAccessFile(String filename, String mode)` mode "r"
- `void seek(long position)`
- `long getFilePointer()`
- `long length()`
- `readByte()/writeByte()` `readShort()/writeShort()`
- `readInt()/writeInt()` `readLong()/writeLong()` 1 2 4 8

4.2.1 Endianness

- **Big Endian** 1000000 [0, 15, 66, 64]
- **Little Endian** 1000000 [64, 66, 15, 0]

Java `RandomAccessFile`

WAV BMP

`Integer.reverseBytes()` `Short.reverseBytes()`

4.2.2 WAV

WAV

1. RIFF 12 "RIFF" "WAVE"
2. fmt 24
3. data

WAV

WAV

```

void reverse_wav(String filename) throws IOException {
    RandomAccessFile f = new RandomAccessFile(filename, "rw");
    // 跳转到data块大小字段，读取采样总数
    f.seek(40);
    int n = Integer.reverseBytes(f.readInt()) / 2;
    // 读取所有采样
    ArrayList<Short> samples = new ArrayList<Short>();
    for (int i = 0; i < n; i++) {
        samples.add(Short.reverseBytes(f.readShort()));
    }
    // 跳回数据起始位置，倒序写入
    f.seek(44);
    for (int i = 0; i < n; i++) {
        f.writeShort(Short.reverseBytes(samples.get(n - 1 - i)));
    }
    f.close();
}

```

4.3

4.3.1

`close()``try-finally`

```

FileInputStream fis = new FileInputStream(filename);
try {
    fis.read();
} catch (IOException e) {
    // 处理错误
} finally {

```

```
    fis.close();
}
```

close

4.3.2 try-with-resources

Java try-with-resources

AutoCloseable

```
try (FileInputStream fis = new FileInputStream(filename)) {
    fis.read();
} catch (IOException e) {
    // 处理错误
}
```

fis.close()

4.4 Tutorial 10 BMP

4.4.1 BMP

BMP

- BMP 14 "BM"
- DIB 40
- BGR
- 4 0 padding = (4 - (width * 3) % 4) % 4

4.4.2

BMP

```
private int readLittleEndianIntFromStream(FileInputStream fis) throws
IOException {
    int b1 = fis.read(); // 最低位字节
    int b2 = fis.read();
    int b3 = fis.read();
    int b4 = fis.read(); // 最高位字节
    return (b4 << 24) | (b3 << 16) | (b2 << 8) | b1;
}
```

- << b2 << 8 8
- | int

4.4.3

1. **Grayscale** $\text{gray} = 0.299 \times R + 0.587 \times G + 0.114 \times B$

```
public BMPImage toGrayscale() {
    byte[] newData = new byte[pixelData.length];
    for (int i = 0; i < pixelData.length; i += 3) {
        int b = pixelData[i] & 0xFF;
        int g = pixelData[i+1] & 0xFF;
        int r = pixelData[i+2] & 0xFF;
        int gray = (int)(0.299 * r + 0.587 * g + 0.114 * b);
        newData[i] = (byte) gray;
        newData[i+1] = (byte) gray;
        newData[i+2] = (byte) gray;
    }
    return new BMPImage(...);
}
```

$\& 0xFF$ byte 0~255

2. **Negative** $\text{newValue} = 255 - \text{原值}$
3. delta 0~255
4. 3×3

4.4.4

- 1920×1080 620 IO
- 2 IO

Inheritance

Lecture 11 + Tutorial 11

5.1

- Superclass
- Subclass
- Object

"is-a"

Dog Mammal

Java extends

```
class Animal {}
class Mammal extends Animal {}
class Dog extends Mammal {}
```

extends

Object

5.2 Liskov Substitution Principle

```
/
int age(Animal a)    Dog Cat    Animal
```

5.3 Inheritance

```
class Button {
    void setCaption(String caption) {...}
    String getCaption() {...}
}
class AcceptButton extends Button {
    void accept() {...}
}
```

AcceptButton

Button

setCaption

getCaption

accept

5.4 Method Overriding

```
class BasicRobot {
    void speak() {
        System.out.println("Hi, I'm a robot!");
    }
}
class CleaningRobot extends BasicRobot {
    @Override
    void speak() {
        System.out.println("Hi, I'm a cleaning robot!");
    }
}
```

- @Override

- `CleaningRobot`
`ArrayList<BasicRobot>` `speak()`
`final`

5.5 Protected

Java

- `private`
- `protected`
- `public`

```
class Counter {
    protected int value;
    void increase() { this.value++; }
}
class DecreaseableCounter extends Counter {
    void decrease() { this.value--; }
}
```

`value` `protected` `DecreaseableCounter`

5.6 Abstract Class

`new`

`abstract`

```
abstract class Animal {
    abstract String voice();
    void speak() {
        System.out.println(this.voice());
    }
}
class Dog extends Animal {
    String voice() {
        return "woof";
    }
}
```

`Animal` `voice()` `Dog` `voice()` `speak()`

5.7 super

super

1. `super(参数)`
2. `super.方法名(参数)`

```
class ImageWithUndo extends Image {
    ImageWithUndo(int width, int height) {
        super(width, height);
        this.history = new Stack<UndoAction>();
    }
    void set(int i, int j, Color color) {
        history.push(new UndoAction(i, j, color));
        super.set(i, j, color);
    }
}
```

-
- `set` `set` `super.set` `set`
 `this.set`

5.8 Tutorial 11

5.8.1 Object

Object

- `toString()`
- `equals()`
- `hashCode()`

```
public class Student {
    private String name;
    private int id;

    @Override
    public String toString() {
        return "Student{name=" + name + ", id=" + id + "}";
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
    }
}
```

```
        if (!(obj instanceof Student)) return false;
        Student other = (Student) obj;
        return this.name.equals(other.name) && this.id == other.id;
    }
}
```

5.8.2

Java	Throwable	Exception	Error
IOException	RuntimeException	Exception	
Exception			

Interface

Lecture 12 + Tutorial 12

6.1

Java

fly

6.2

implements

```
interface Flying {
    void fly();
}

class Bat extends Animal implements Flying {
    public void fly() { /* 飞行实现 */ }
}

class Airplane extends Vehicle implements Flying {
    public void fly() { /* 飞行实现 */ }
}
```

public

public

```
void makeThemFly(ArrayList<Flying> flyingObjects) {
    for (Flying f : flyingObjects) {
        f.fly();
    }
}
```

Flying

6.3

6.3.1

Java Iterable Iterator

```
interface Iterator<T> {
    T next();
    boolean hasNext();
}
interface Iterable<T> {
    Iterator<T> iterator();
}
```

Iterable

for-each

for-each

iterator()

hasNext next

6.3.2 Visitor Pattern

"

"

"

"

```
interface TreeVisitor<T> {
    void visitNode(T root);
    void visitEmptyTree();
}
abstract class Tree<T> {
    abstract void accept(TreeVisitor<T> visitor);
}
```

accept

visitor

visitor

6.4 Functional Interface

```
interface UnaryFunction {
    double apply(double x);
}
```

map

```
ArrayList<Double> map(UnaryFunction f, ArrayList<Double> a) {
    ArrayList<Double> res = new ArrayList<Double>();
    for (double x : a) {
        res.add(f.apply(x));
    }
    return res;
}
```

6.5 Lambda

Lambda

```
System.out.println(map(x -> Math.sin(x), a));
```

x -> Math.sin(x)

Lambda

sin(x)

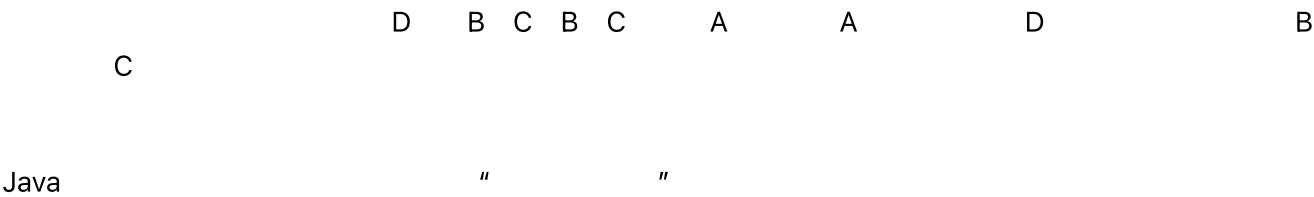
Lambda

UnaryFunction

apply

6.6 Tutorial 12

6.6.1 Diamond Problem



6.6.2 vs

extends	implements
"is-a"	"can-do"